



摘要：

黄仁勋表示，AI编程正从“写Prompt”转向“设计Loop”，即构建自动化循环系统，让AI自主迭代、验收并完成任务，人类退居规则制定者。Claude Code、OpenAI Codex已落地该范式。但专家提醒，警惕Token成本与“理解力债务”，AI可外包思考，却无法替代人类对问题的真正理解。

Prompt已死，loop当立。

这就是最近网上热传热议，然后老黄黄仁勋给AI新趋势画的新重点：

Nobody writes prompts anymore. The new job is to write and handle loops.（现在根本没有人写Prompt了，新时代的核心工作是编写和管理loop。）



啥是loop？这个词直译过来是“循环”，换成AI圈的说法就是：

你不再亲手给AI下指令，而是设计一个系统，让系统替你下指令、替你验收、不合格自己重来，直到活干完。

嗯？这不就是如今Agent那一套吗？为啥又搞个新概念出来？

暂且按下此疑惑不表，待我环顾一圈后发现，这个“loop”还真挺火——

除了老黄，“龙虾之父”Peter、“Claude Code之父”Boris Cherny、吴恩达等一众大佬全都在谈、在大力推loop。

（Peter）别再给编程Agent写提示词了，去设计循环，让循环替你提示Agent。



(Boris) 我已经不给Claude写提示词了。我有一堆循环在跑，是它们在给Claude下指令、决定下一步做什么。我的工作，就是写循环。



而当“写loop”取代“写prompt”成为大佬们新的日常，loop显然已经越过了“又一个新概念”的阶段。

剩下的问题就只有：

loop具体是指什么？它怎么就突然火起来了？

loop到底是什么

要理解loop这个新东西，我们得先回顾一下之前的那套旧范式。

过去两年AI编程的标准动作是这样的：

你写一条prompt，AI吐一段代码，你看了不满意，再写一条，AI再改，你再看……

反正就是来回拉锯，人全程盯着。

卡帕西之前还侧面吐槽了“人就是瓶颈”这件事，而且劝告大家：

你不能坐在那里等着给每一步写prompt，你得把自己从流程中抽离出来。



Skill Issue: Andrej Karpathy on Code Agents, AutoResearch, and the Loopy Era of AI



No Priors: AI, Machine Learning, Tech, & Startups
8.62万位订阅者

订阅



1.6万



公众号



量子位



分享



把人从流程中抽离出来，这正是loop要解决的事。

其核心逻辑只有一句话：

你定义一个目标，AI自己跑，跑完自己验收，不合格带着报错再来一轮，直到通过或者撞上预算上限才停。

此时人的角色就从“传话人”变成了“规则设计者”。

所以回到开头的疑问：这跟Agent有什么区别？

显而易见，Agent是干活的那个人，而loop是让这个人不用你盯着也能持续干活的那套管理机制。

没有loop的Agent，你提一句它动一下，本质上还是个听话的工具。

套上loop的Agent，才真正变成了一个能自转的系统。



原理听起来确实不复杂，但貌似仍有点抽象。

别急，我又去翻了下当前loop的实际落地情况，结果发现它其实已经藏在了我们熟悉的系统里。

围绕loop，产品落地层目前已经形成了“双雄对峙”格局。

一个就是大家天天都在用的Claude Code，它围绕loop做了三件套：

`/loop`负责定时循环，`/goal`负责目标驱动（跑到验收条件满足为止），`/schedule`负责云端定时任务（合上电脑也能跑）。

其中最精妙的设计是`/goal`，它背后藏着loop最关键的一条原则——自己不能判自己的卷子。

Claude Code把这条原则直接写进了产品架构：

写代码的是大模型，验收的是另一个独立的小模型Haiku，两个模型各司其职。

这样一来，Agent不会自己给自己打高分，验收才有真实的约束力。



另一个就是OpenAI Codex。

Codex的玩法更接近“自动化流水线+目标驱动+多个子Agent”的组合，在一些开发者的实际体验中，能看到最多8个Agent同时跑在各自的云端沙箱里，各干各的活，最后把结果汇总回来。

有意思的是，虽然两家的实现路径不太一样，但最终长出来的形态高度相似——

都是把复杂任务拆碎，分给多个Agent并行去跑，再统一汇总。

在公开评测和社区口碑里，两者的表现也已经非常接近。

这也说明一个问题，**模型本身已经卷不出太大差别了，真正的差距在上层的loop编排。**

说到这儿，咱们直接看看“Claude Code之父”Boris Cherny每天怎么工作的就全明白了。

他自述去年11月卸载了IDE，一个月没打开过，索性删了。

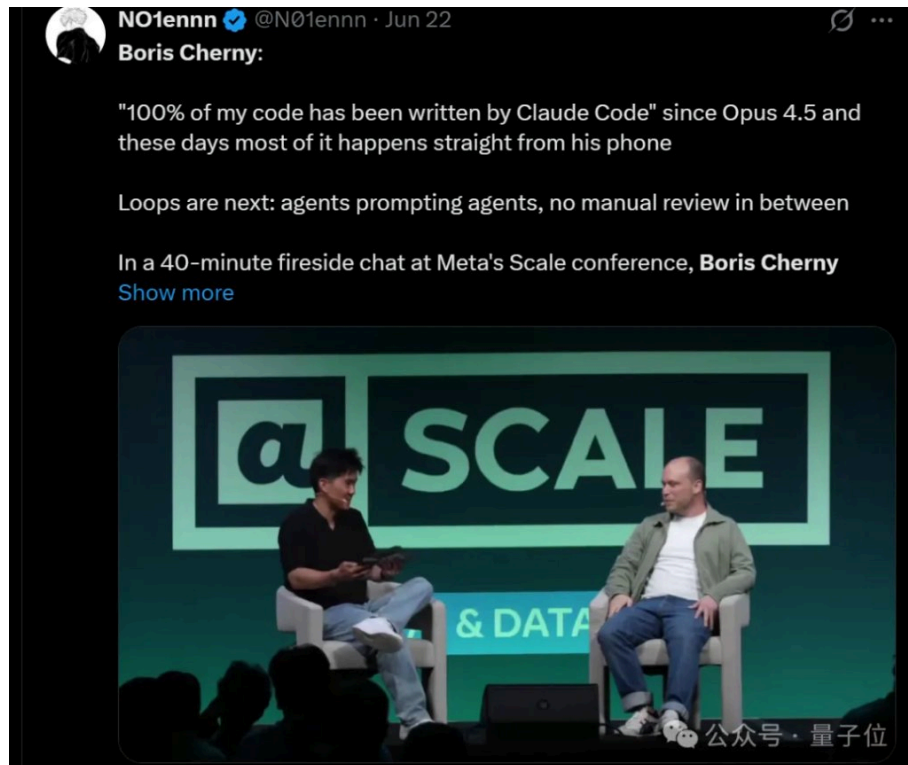
现在他手下几百个小Agent同时跑，有的扫GitHub issue，有的读Slack上的用户反馈，有的监控CI失败。每个Agent在自己隔离的代码分支里干活，一个写代码，另一个跑测试验收。

搞不定的才进他的收件箱，等他来做判断。



据他透露，自Opus 4.5以来，其所有代码都是Claude Code写的，如今大部分代码都是直接在他的手机上完成。

接下来是循环，Agent之间互相提示，中间无需人工审核。



看到没，loop的终极形态已经很清晰了：

人不写代码，也不写prompt，只写规则和判断，剩下的全交给loop。

怎么loop起来

那么，我们该怎么loop起来呢？

X上有叫Codez的博主已经都替大家总结好了，他发了一份14步实操roadmap，这里我挑了一些干货。



step 1: 先别急着建，先做“4条件测试”

loop不是什么活儿都能往里面塞，瞎建只会亏钱。

在动手之前，先回答四个问题：

- 任务重复发生吗？
- 有自动化验收手段吗？
- Token预算扛得住吗？
- Agent有“高级工程师”的工具吗？

维度	核心问题	如何 loop 起来（思路）	关键工具 / 示例
 任务重复发生吗？	至少每周来一次。 一次性任务直接写 prompt 更快更便宜	- 明确重复的触发条件（如：每周一、有新 issue、CI 失败时） - 把重复动作固化为自动化流程 - 让系统自动触发任务，而不是人手动发起	示例： CI 每晚扫描失败 工具： GitHub Actions、Cron、Webhooks、Claude Code / Codex 的 /schedule
 有自动化验收手段吗？	测试套件、类型检查、Lint、构建工具——有没有东西能自动判断“干得好不好”	- 为任务定义可自动验证的标准 - 结果必须可被机器判定（通过/失败） - 失败时自动进入修复循环	示例： 运行测试通过才算完成 工具： Jest、PyTest、TypeScript 类型检查、ESLint、Build / Compile
 Token 预算扛得住吗？	loop 会反复读上下文、重试、探索，烧 token 比你想象的多	- 预估单次循环的 token 消耗 - 设置最大循环次数 / 预算上限 - 超限时自动停止并通知	示例： 单次循环约消耗 50k tokens 工具： Anthropic Usage API、OpenAI Usage API、自定义预算监控脚本
 Agent 有“高级工程师”的工具吗？	能看日志、能复现环境、能跑自己写的代码	- 赋予 Agent 完整的环境访问权限（读取日志、执行命令、运行测试） - 提供足够的上下文与工具集 - 让 Agent 像资深工程师一样排查问题	示例： Agent 执行命令并读取日志 工具： MCP（文件、终端、数据库、浏览器等）、Docker、云沙箱环境、GitHub API
⚠️ 不满足以上任一条件的场景 ✖️ 一次性任务 / 探索性工作 ✖️ 没有可自动验收的标准 ✖️ 预算有限，容易超支 ✖️ 工具受限，无法复现问题 → 用 prompt 更简单、更便宜			
✅ 适合 loop 的典型场景（举例） 🔄 CI 失败自动修复 扫描失败，分类原因，生成修复 PR 🔄 依赖更新 定期检查依赖更新，测试兼容性，开 PR 🔄 代码质量修复 自动执行 Linter / 格式化，修复问题并提交 🔄 Flaky 测试排查 反复运行，收集日志，定位不稳定用例 🔄 Issue → PR 草稿 从 Issue 生成实现方案，通过测试后提交 PR			

△图片由AI生成

四个全过，才值得建loop。

step 2：从最小可行loop开始

第一次别搞花活，就建一个四件套：

- 一个触发器（Automation）：定时跑、事件触发跑都行。Claude Code里用/loop，Codex里用Automations面板。
- 一个技能（Skill）：把项目上下文写进STATE.md，让每次运行不用重新解释一遍。
- 一个状态文件（State File）：用Markdown记下“做到哪了、什么成了、什么挂了”，下次接着跑。
- 一个门禁（Gate）：测试、类型检查、构建——能自动拦住坏结果的东西。

而且顺序很关键：先手动跑通一次 → 写成Skill → 包进loop → 最后才上定时。

跳步是loop死在生产环境的主要原因。

step 3：做“拆卷子”的人，别做“判卷子”的

整个loop设计里最重要的一条原则前面已经提过了——写代码的和验代码的，必须分开。

落地到具体操作就是：

用一个模型（或者子Agent）负责写，用另一个独立的模型（或者子Agent）负责验收，验收的那个不能看到写代码的那个的推理过程。





为什么这很重要？因为模型给自己写的代码打分时，往往“手太松了”。

所有“看起来不错”的代码，在独立验收器面前大概率能挑出一堆毛病。

step 4：别人踩过的坑就别再踩了

附几个避坑指南。

- 1、没有硬停止条件。** loop跑到你发现账单或者被限流才停，所以需要设Token上限、迭代次数上限、时间限制。
- 2、状态不落地。** Agent的记忆是短时的，今天学到的东西明天就忘了，所以需要写进状态文件 ([STATE.md](#))，每次运行接着读。
- 3、让loop碰“需要判断”的活。** 架构重写、鉴权代码、支付逻辑、产品方向决策，这些别让loop碰。loop适合干“对错清晰、机器可验证、不依赖人的判断”的活，比如Lint自动修复、依赖更新PR、CI失败分类、Flaky测试复现。
- 4、不读Diff。** loop合入代码越来越快，你对代码库的理解越来越浅。这叫“理解力债务”——真正的代价不是Token账单，而是某天你要调试一个团队里没人读过的系统。所以建议你读Diff，哪怕只是扫一眼。

step 5：衡量指标就一个

别管烧了多少Token、开了多少PR、跑了多少次任务。

唯一有用的指标是：每个被接受的改动，平均成本是多少。

如果你的“被接受率”低于50%，说明你在做loop本该替你省掉的评审工作，即loop在亏钱。

从提示词到loop，四次范式跃迁

原理和方法搞懂了，最后的问题只有一个：

loop为什么现在火了？

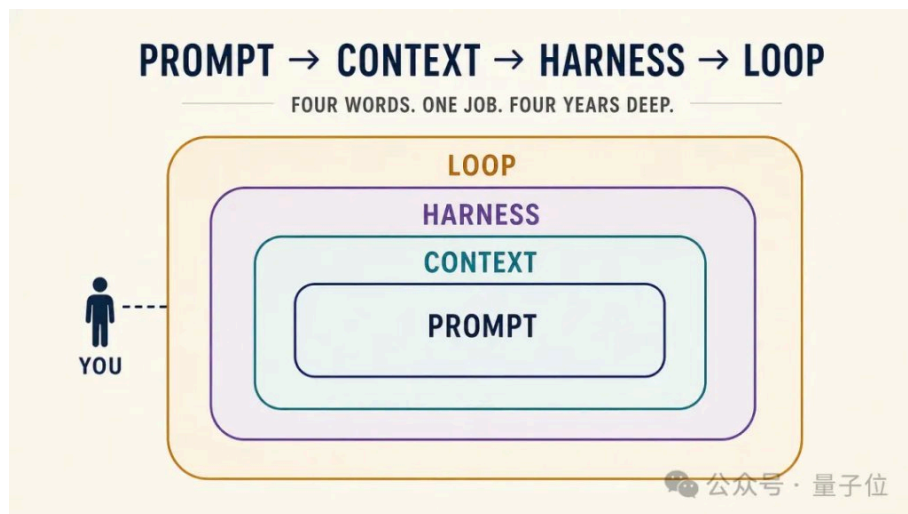
虽然严格来说，loop Engineering这个概念只有不到三周的历史。

但它不是凭空蹦出来的，往回拉一下时间线就能看到一条很清晰的演化路径。

这条路径大佬们也已经替大家总结好了，咱直接抄作业：



从Prompt → Context → Harness → loop，一共四次。



简单来说，从2023~2024年，这是**Prompt Engineering**的天下。

当时大家都在琢磨一件事：提示词怎么写才能让AI好好干活。

写得好和写得不好，出来的东西天差地别，所以那会儿“会不会写prompt”基本就等于“会不会用AI”。

这一阶段，人和AI的关系还停在最表面那层——你说一句，它回一句，细到每一个指令都要人亲自敲。

但随着模型能力增强、上下文窗口变长，以及RAG和代码库接入逐渐普及，问题开始发生第一次迁移。

大约在2024到2025年前后，行业开始强调“**Context Engineering**”的重要性，关注点从“怎么问”变成了“给AI看什么”。

也就是说，AI不再只依赖一句提示词，而依赖你提供的整个背景。

这一阶段，信息组织能力开始比写prompt更重要，控制粒度从“一句话”上移到了“一堆信息”。

到了2025~2026年，随着Agent系统逐步进入真实开发流程，问题继续往外扩展。

这时人们发现，光给信息和上下文也不够了，AI得能接工具、能跑代码、能调接口、能走权限审批。

因此，你得给它搭一个能干活、能约束、能调取真实世界资源的运行环境。

“**Harness Engineering**”正是为此而生。

而在Harness的基础上，“**loop Engineering**”成了最新的进化方向。

如果说Harness解决的是“AI能不能在真实环境里干活”的问题，那loop解决的就是“AI能不能在这个环境里持续干活、自己推进任务、不需要人一步步盯着”的问题。

它的核心不再是单次执行能力，而是一个闭环系统的运行能力。



所以从Prompt到Context，再到Harness，再到loop，看起来像是概念的更替，但本质上是一条连续的迁移路径：

人类对AI的控制粒度不断上移，从“写一句话”，变成“提供信息”，再变成“搭建系统”，最终变成“设计循环”。

一个逐渐解放人类双手的过程。

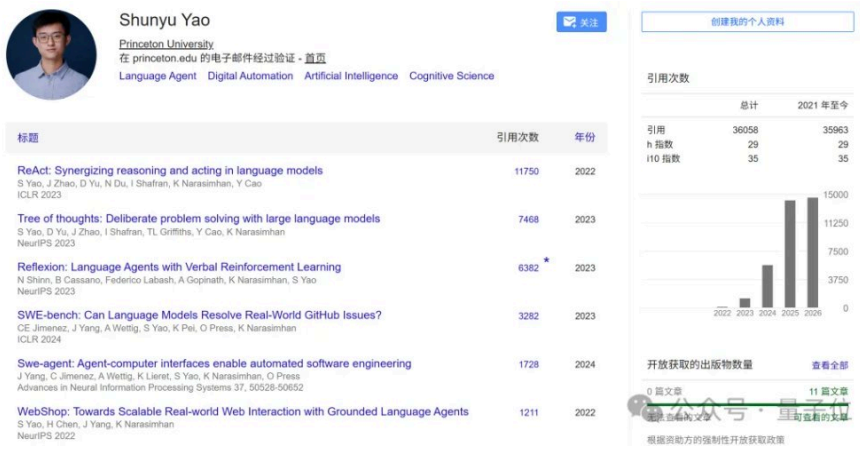


实际上，虽然loop这玩意儿刚在工业界火起来，但学术界其实早就有了类似理念。

而且很多重要工作都和我们今天熟悉的一个人有关：姚顺雨（腾讯那个）。

他在大模型Agent方向最具代表性的工作之一，是2022年的ReAct框架（Reason+Act）。

这篇工作在ICLR 2023拿到Oral级别，也在后续获得了上万引用量。



ReAct做了一件很关键的事：把“推理”和“行动”绑定成一个循环过程。

大模型不再是一次性输出答案，而是先进行可解释的思考，再调用工具执行动作，执行之后再观察环境反馈，然后进入下一轮推理。抽象出来就是：

思考 → 行动 → 观察 → 再思考 → 再行动...

这个结构本质上就是一个最早被系统化表达的“agent loop”雏形。

在ReAct之后，这条路线被不断扩展，比如Reflexion引入“从错误中学习”的反馈机制，Tree of Thoughts扩展成多路径搜索式推理，以及后续一系列tool-use agent工作逐步完善“规划+执行+反馈”的完整链路。

这些学术成果一点一点往前拱，最终才在工程界收敛成今天所说的“loop系统”。

所以从学术视角看，loop不是某一个人的发明，它是一条逐步收敛的技术路径。

只不过在这条路上，恰好有个我们熟悉的华人站在了一个关键节点上。

最后不得不感慨，从Prompt到loop，AI的发展还是太快了。

太快带来的后果是，有人兴奋激动，也有人难掩担忧。

而loop Engineering的命名者、Google工程主管Addy Osmani，正是后者当中的一员。

他在《loop Engineering》这篇长文里写得很明白：

还很早期。我持保留态度。token成本你必须非常小心。

Loop Engineering

JUNE 7, 2026

Loop engineering is replacing yourself as the person who prompts the agent. You design the system that does it instead. A loop here can be thought of a recursive goal where you define a purpose and the AI iterates until complete. I believe this may be the future of how we work with coding agents. However, its still early, I'm skeptical and you absolutely *have* to be *careful* about token costs (usage patterns can vary wildly if you are token rich or poor), so I want to unpack what it is and what it means.

公众号 · 量子位

卡帕西的话更让人深思，他在红杉资本AI Ascent 2026大会上引用过一句让他本人反复回想的话：

你可以外包你的思考，但你没法外包你的理解。

翻译翻译，AI可以替你想办法，但你自己得真懂问题本身。

这大概是整场loop热潮里最清醒的声音。

本文来源：[量子位](#)



风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

136 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

1 条评论

空

12小时前 中国浙江省

越搞越复杂

0 回复